



VALIDACIONES CON SPRING



✓ Lección: Validaciones en Formularios con Spring Boot + Thymeleaf

🔧 Paso 1: Agregar las librerías de validación de Spring

📁 Abrimos: `pom.xml`

Agregamos lo siguiente para que se agreguen las librerías de validaciones de Spring:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

🔧 Paso 2: Agregar anotaciones en la entidad `Persona`

📁 Abrimos: `src/main/java/gm/domain/Persona.java`.

Agregamos las siguientes anotaciones:

```
@NotEmpty
private String nombre;

@NotEmpty
private String apellido;

@NotEmpty
@email
private String email;

// No validamos teléfono (puede quedar vacío)
```

🧠 @NotEmpty valida que no sea null ni cadena vacía.

✉️ @Email valida el formato correcto de correo electrónico.

💻 Paso 3: Mostrar errores en la vista `modificar.html`

📁 Abrimos: `src/main/resources/templates/modificar.html`.

Después de cada `<input>`, agregamos validaciones con Thymeleaf:

```
<span th:if="${#fields.hasErrors('nombre')}}" th:errors="*{nombre}">Error en nombre</span>
```

Repetimos para el campo de `apellido`.

```
<span th:if="${#fields.hasErrors('apellido')}}" th:errors="*{apellido}">Error Apellido</span>
```

Repetimos para el campo de `email`.

```
<span th:if="${#fields.hasErrors('email')}}" th:errors="*{email}">Error Email</span>
```

💡 También cambiamos el tipo del campo email de `type="email"` a `type="text"` para que no interfiera la validación de HTML y se muestre el mensaje de Spring:

```
<input type="text" name="email" th:field="*{email}" />
```

🔄 Paso 4: Validar en el controlador

📁 Abrimos: `src/main/java/gm/web/ControladorInicio.java`.

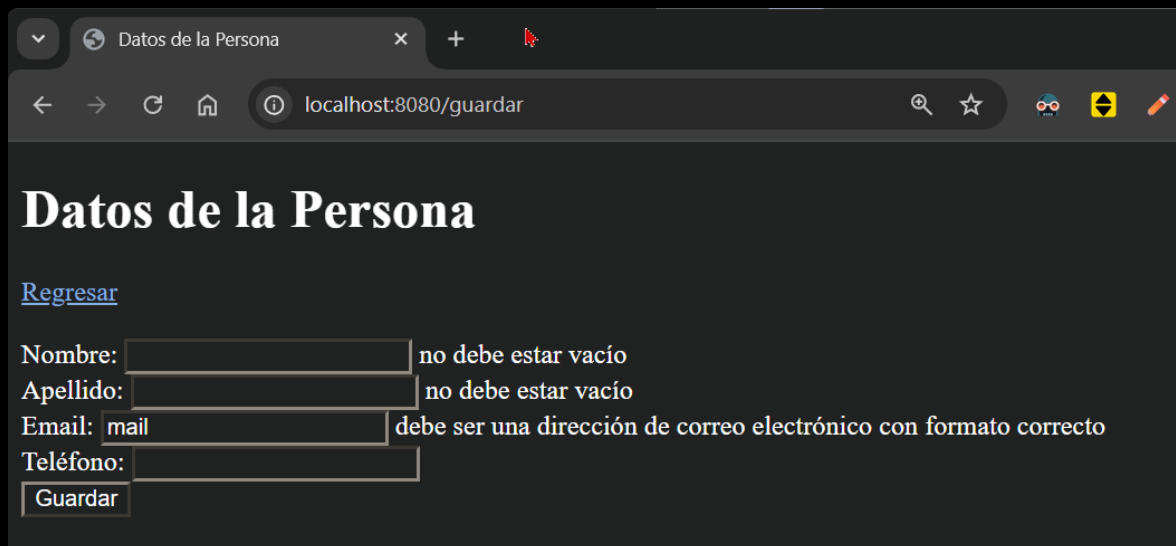
Modificamos el método `guardar` así:

```
@PostMapping("/guardar")
public String guardar(@Valid Persona persona, Errors errores) {
    if (errores.hasErrors()) {
        return "modificar";
    }
    personaService.guardar(persona);
    return "redirect:/";
}
```

💬 Si hay errores, regresamos al formulario y se muestran los mensajes.

🔧 Paso 5: Probar en el navegador

1. Ir a `localhost:8080`.
2. Crear o editar una persona.
3. Dejar nombre, apellido o email vacíos → ❌ Aparecerán errores.
4. Probar con email inválido → ❌ Verás el mensaje "no es una dirección de correo bien formada".
5. Ingresar valores válidos → ✅ Se guarda exitosamente.



The screenshot shows a web browser window with the title 'Datos de la Persona'. The address bar shows 'localhost:8080/guardar'. The page content includes a 'Regresar' link and a form with the following fields and error messages:

- Nombre: no debe estar vacío
- Apellido: no debe estar vacío
- Email: debe ser una dirección de correo electrónico con formato correcto
- Teléfono:
- Guardar

🧠 Conclusión

Con este ejercicio implementamos:

- ✓ Agregamos las librerías de validación de Spring en el archivo `pom.xml`
- ✓ Validaciones en el backend con `javax.validation`
- ✓ Mensajes de error en la vista con `Thymeleaf`
- ✓ Validación del email con formato
- ✓ Control de errores desde el controlador

Todo en una arquitectura profesional MVC con Spring Boot 🌱.

✂ Código completo `pom.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.4</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>gm</groupId>
    <artifactId>hola-spring</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>hola-spring</name>
    <description>Hola Mundo con Spring</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
```

```
<properties>
  <java.version>21</java.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

```
        </plugins>
    </build>

</project>
```

✂ Código completo Persona.java

```
package gm.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.io.Serializable;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotEmpty;

@Entity
@Table(name="persona")
public class Persona implements Serializable{

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idPersona;

    @NotEmpty
    private String nombre;

    @NotEmpty
    private String apellido;

    @NotEmpty
    @Email
    private String email;

    private String telefono;

    public Persona() {
    }
}
```

```
public Persona(Long idPersona, String nombre, String apellido, String email, String telefono) {
    this.idPersona = idPersona;
    this.nombre = nombre;
    this.apellido = apellido;
    this.email = email;
    this.telefono = telefono;
}

public Long getIdPersona() {
    return idPersona;
}

public void setIdPersona(Long idPersona) {
    this.idPersona = idPersona;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getTelefono() {
    return telefono;
}

public void setTelefono(String telefono) {
```

```

        this.telefono = telefono;
    }

    @Override
    public String toString() {
        return "Persona{" + "idPersona=" + idPersona + ", nombre=" + nombre + ", apellido=" + apellido +
            ", email=" + email + ", telefono=" + telefono + '}';
    }
}

```

✖ Código completo modificar.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns="http://www.w3.org/1999/xhtml">
    <head>
        <title>Datos de la Persona</title>
        <meta charset="UTF-8"/>
    </head>
    <body>
        <h1>Datos de la Persona</h1>
        <a th:href="@{/}">Regresar</a>
        <br/>
        <form th:action="@{/guardar}" method="post" th:object="${persona}">
            <input type="hidden" name="idPersona" th:field="*{idPersona}"/>
            <br/>
            <label for="nombre">Nombre:</label>
            <input type="text" name="nombre" th:field="*{nombre}"/>
            <span th:if="${#fields.hasErrors('nombre')}" th:errors="*{nombre}">Error Nombre</span>
            <br/>
            <label for="apellido">Apellido:</label>
            <input type="text" name="apellido" th:field="*{apellido}"/>
            <span th:if="${#fields.hasErrors('apellido')}" th:errors="*{apellido}">Error Apellido</span>
            <br/>
            <label for="email">Email:</label>
            <input type="text" name="email" th:field="*{email}"/>
            <span th:if="${#fields.hasErrors('email')}" th:errors="*{email}">Error Email</span>
            <br/>
            <label for="telefono">Teléfono:</label>
            <input type="tel" name="telefono" th:field="*{telefono}"/>
            <br/>
            <input type="submit" name="guardar" value="Guardar"/>
        </form>
    </body>
</html>

```

```
</body>
</html>
```

✖ Código completo ControladorInicio.java

```
package gm.web;

import gm.domain.Persona;
import gm.servicio.PersonaService;
import jakarta.validation.Valid;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.Errors;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class ControladorInicio {

    private static final Logger log = LoggerFactory.getLogger(ControladorInicio.class);

    @Autowired
    private PersonaService personaService;

    @GetMapping("/")
    public String inicio(Model model) {
        log.info("Ejecutando el controlador Spring MVC");
        var personas = personaService.listarPersonas();
        model.addAttribute("personas", personas);
        return "index";
    }

    @GetMapping("/agregar")
    public String agregar(Persona persona) {
        return "modificar";
    }

    @PostMapping("/guardar")
    public String guardar(@Valid Persona persona, Errors errores){
        if(errores.hasErrors()){
            return "modificar";
        }
    }
}
```

```
    }  
    personaService.guardar(persona);  
    return "redirect:/";  
}  
  
@GetMapping("/editar/{idPersona}")  
public String editar(Persona persona, Model model) {  
    persona = personaService.encontrarPersona(persona);  
    model.addAttribute("persona", persona);  
    return "modificar";  
}  
  
@GetMapping("/eliminar")  
public String eliminar(Persona persona) {  
    personaService.eliminar(persona);  
    return "redirect:/";  
}  
}
```

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)